

URL Shortener Project Explanation

What This Project Does

This project is a Laravel and React based URL shortener.

The main purpose is simple:

1. A user enters a long URL.
2. The app generates a short random code.
3. The app creates a short link.
4. When someone opens the short link, the app redirects them to the original long URL.
5. The app records click analytics in the background.

Example:

Long URL:

```
'''txt
https://example.com/some/very/long/url/with/query/params
'''
```

Short URL:

```
'''txt
https://yourdomain.com/J7qudkrM
'''
```

What To Say About The Project

This is a full-stack URL shortener application built with Laravel for the backend and React for the frontend.

The user can paste a long URL, click the shorten button, and the system generates a short 8-character link. When that short link is opened, Laravel finds the original URL in the database and redirects the user to it.

The project also tracks clicks using a queue-based system so that redirects stay fast even when many users are clicking links.

Why Local Links Show 127.0.0.1 Or 8001

During development, the Laravel server runs locally on the computer.

That is why links may look like:

```
'''txt
http://127.0.0.1:8001/J7qudkrM
'''
```

Here:

- '127.0.0.1' means the local computer.
- '8001' is the local Laravel development server port.

- 'J7qudkrM' is the short code.

This is normal during local development.

What Happens After Hosting

After buying a domain and hosting the project publicly, the link will not show '127.0.0.1' or '8001'.

It will look like:

```
'''txt
https://yourdomain.com/J7qudkrM
'''
```

or with a short domain:

```
'''txt
https://lnksrt.in/J7qudkrM
'''
```

Public HTTPS websites use port '443', and browsers hide that port automatically.

So users will only see the clean domain and short code.

Can The Domain Be Removed Completely?

No, a public URL cannot work without a domain.

This will not work properly:

```
'''txt
http://J7qudkrM
'''
```

Because the browser will think 'J7qudkrM' is the domain name.

A real short link must have:

```
'''txt
https://domain.com/code
'''
```

Example:

```
'''txt
https://lnksrt.in/J7qudkrM
'''
```

Will Local Links Work On Mobile?

A local link like this will not work on mobile:

```
'''txt
```

```
http://127.0.0.1:8001/J7qudkrM
'''
```

Because on the phone, '127.0.0.1' means the phone itself, not the laptop.

For mobile testing on the same Wi-Fi, use the laptop's local IP address and run Laravel li this:

```
'''bash
php artisan serve --host=0.0.0.0 --port=8001
'''
```

Then the mobile link may look like:

```
'''txt
http://192.168.1.25:8001/J7qudkrM
'''
```

For real public access from anywhere, the app must be hosted with a real domain.

Backend Flow

When a user creates a short link:

```
'''txt
React frontend
-> POST /api/urls
-> Laravel UrlController
-> ShortCodeGenerator creates 8-character code
-> Save original URL and short code in database
-> Return short URL to frontend
'''
```

When someone opens a short link:

```
'''txt
Browser opens /J7qudkrM
-> Laravel RedirectController
-> Find short code in cache or database
-> Dispatch click tracking job
-> Redirect user to original URL
'''
```

Database Tables

The 'urls' table stores:

- Original long URL
- Short code
- Click count
- Active or paused status
- Expiry date
- Last clicked time

The 'clicks' table stores:

- URL ID
- Hashed IP
- Hashed user agent
- Referrer
- Click timestamp

Why Queue Is Used

Click tracking is done using a queue.

This keeps redirects fast.

Instead of making the user wait while analytics are saved, the app redirects immediately and records analytics in the background.

For local development, the project can use the database queue.

For production, Redis can be used for better performance.

Redis And Kafka

Redis is used for:

- Faster cache
- Faster queue processing
- Better performance when many users are clicking links

Kafka-style event publishing is included as a clean architecture layer.

Right now, click events are logged.

Later, a real Kafka producer can be connected if the project needs event streaming at large scale.

Frontend

The frontend is built with React.

The main screen is simple:

1. Paste a long URL.
2. Click Shorten.
3. Copy the generated short link.
4. View recent shortened links.

Important Commands

Run Laravel server:

```
'''bash
php artisan serve --port=8001
'''
```

Run queue worker:

```
'''bash
php artisan queue:work --queue=clicks,default
'''
```

Run migrations:

```
'''bash
php artisan migrate
'''
```

Run tests:

```
'''bash
php artisan test
'''
```

Build frontend:

```
'''bash
npm run build
'''
```

Final Summary

This project is a working full-stack URL shortener.

It can generate short links, redirect users, record click analytics, and support production scaling concepts like caching, queues, Redis, and Kafka-style events.

During local development, links show '127.0.0.1' or a local port like '8001'.

After hosting with a real domain, the links will look clean:

```
'''txt
https://yourdomain.com/J7qudkrM
'''
```